

Algorithms and Programs

TC1037 · Algorithmic Thinking · Lecture 01

Dr. Leopoldo Rodríguez Salazar

Tuesday 22 September 2026

Tecnológico de Monterrey · Campus Querétaro · Group 510

Tell me how to draw a square.

I will follow your instructions **exactly**.

What just happened

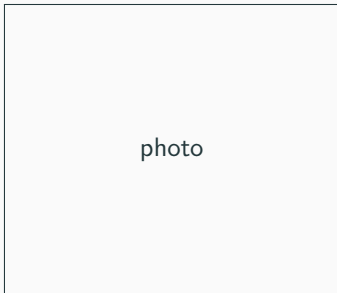
- Every instruction you gave me was **correct**.
- I produced garbage.
- I did not misunderstand you — I understood you **too literally**.

What was missing was never the *idea*.
It was precision about the things you thought were obvious.

Who is talking to you

Leopoldo Rodríguez Salazar — *Leo*

- PhD; aerospace and automotive engineering.
- Jaguar Land Rover · GE Aviation · Ford.
Marie Curie Fellow.
- Today: CEO of **Helgen Technologies**,
Querétaro — we build telemetry hardware
and the software behind it.
- rodriguez.l@tec.mx · Canvas Inbox



*Asesorías: Tue 16:30–17:30, Room
4206*

I write algorithms for a living. The code has never been the hard part.

The week, every week

Tuesday	17:35–18:55 · Room 4206	quick exam (first 20 min, from week 2) + session
Friday	16:30–17:30 · Room 4206	laboratory with Luis Castañón (TA)
	17:35–18:55 · Room 4206	session
Sunday	23:59 · Canvas	homework due (when there is one)
Tuesday	16:30–17:30 · Room 4206	asesorías — me, in person

Attendance is taken 17:35–17:40. After 17:40 you are absent.
Graded work happens *in the room*. Miss the session, the activity is zero.

Ground rules

- **Paper first.** Every exercise, lab and homework starts with the algorithm written by hand. Code without it is incomplete.
- **No AI in this room or in the lab.** Outside: to *understand*, never to *produce*. You will explain your own code out loud on project deliverables.
- **Laptops closed unless we are typing.** Phones away. Ask anything, any time.
- **Exams:** Respondus LockDown Browser, Trinket editor, no Thonny. **No Macs in exams — if you only have a Mac, tell me Friday.**
- **Deadlines are fixed.** Late work is not received.
- **Integrity:** a breach is a failing grade and goes to the campus committee. Not my call; not negotiable.

How you are graded

4%	Study the levelling module	20%	Midterm exam
1%	Final prior-knowledge diagnostic	15%	Situación problema (project)
12%	Quick exams (5, Tuesdays)	30%	Final exam
10%	Laboratories (10)		
4%	In-class activities (4)	100%	
4%	Homework (8)		

Rubrics are published before each exam — on the course site. Nothing is a secret.

Key dates

Tue 29 Sep	Quick exam 1 (first 20 min) — this week's material
Fri 9 Oct	Final diagnostic of the levelling module (in the lab)
Fri 23 Oct	Midterm exam , in class
Nov 6 · 13 · 20	Situación problema deliverables 1–3 (teams of 2)
Fri 13 Nov	Last day to drop the course
Fri 27 Nov	Situación problema, final delivery (individual)
Tue 1 Dec	Final exam , in class

Each missed or late project stage costs 5 points off the project grade.

The levelling module — 5% and a head start

- An online, adaptive module (*Realizeit*, opened from Canvas): ten short lessons — computer memory, expressions, algorithms, flowcharts, problem solving.
- **Friday 16:30, in the lab:** a 20-minute diagnostic on your laptop. It *unlocks* the lessons and decides which ones you need. Score 100 and you are done.
- Then self-paced: lessons 1–5 by Tue 29 Sep, all ten by **Thu 8 Oct**. Final diagnostic **Fri 9 Oct**, in the lab (1%).
- Before Friday: **bring the laptop**, and set your browser language to Spanish (Mexico) or English — Chrome's auto-translate *off*.

Platform questions: TECservices — TECservices@servicios.tec.mx · WhatsApp 81 1625 5123

A **finite, ordered, unambiguous** sequence of steps that turns a **defined input** into a **defined output**.

Algoritmo

The four properties

Finite It ends. “Keep adding numbers” is not an algorithm. “...until the user types 0” is.

Ordered Break the eggs *then* heat the pan $\neq$ heat the pan *then* break the eggs.

Unambiguous Exactly one reading. If two people get different answers, it is a bug you have not found.

Defined I/O You know what you are given and what you must produce.

Algorithm vs. Program

Algorithm The **thinking**

Natural language, pseudocode, a
flowchart

Read by a human

Program The **typing**

Python

Read by a machine

A program can fail for every reason an algorithm can — **plus syntax**.

Our notation

```
ALGORITHM   average_speed
INPUT:      distance (km), time (hours)
OUTPUT:     speed (km/h)

1. READ distance
2. READ time
3. speed <- distance / time
4. DISPLAY speed
END
```

- <- is assignment — “*gets the value of*”. A command, not a claim.
- Steps are numbered. That is what makes “ordered” checkable.

What happens if the user types 0 for the time?

How to write one that survives

1. State **input** and **output**. Two lines, before anything else.
2. Write the steps, **numbered**. One action per step.
3. **Attack it**. Read it as literally as I read your square.
4. Fix, and repeat.

Step 3 is the one that separates a passing grade from a good engineer.
Everyone can do step 2.

Exercise 1 — Price validation

Individual · on paper · 10 minutes

Write an algorithm that checks whether a price entered by the user is valid.
A price is valid if it is positive.

Then attack it, in writing:

- What does yours do if the user enters exactly 0?
- What if the user enters `hola`?
- Does it say what “valid” *looks like* as output?

Exercise 2 — Metres to feet

Pairs · on paper · 15 minutes

Given a length in metres, calculate and display the equivalent in feet.

1 foot = 12 inches 1 inch = 2.54 cm 1 m = 100 cm

1. Numbered steps, explicit INPUT: / OUTPUT:.
2. **Swap with the pair next to you.** Execute *their* algorithm literally for 3.5 metres.
3. Different answers? Find the step where you diverged. **Write down the word that caused it.**

16:30 – 17:30 Laboratory — this room, with Luis

17:35 – 18:55 Session 2 — me

Bring your laptop, a notebook and a pencil.

The lab opens with a 20-minute diagnostic that needs the laptop.

Everything after it is on paper.

Bring today's algorithm too — you will be attacking each other's.

Eat before you come. Not in the room.

Before Friday

1. Install **Thonny**. Bring it working — we write our first program Friday.
2. Install **Respondus LockDown Browser**; practice exam by Mon 28 Sep, 23:59.
3. Levelling module: nothing yet — it opens Friday with the diagnostic.
4. Read the two handouts: course policies, and *Using AI in this course*.

Only have a Mac? Tell me Friday. Macs are not permitted for exams.

**An algorithm is a specification
a hostile reader cannot misread.**